Binary search: time complexity

Challenge 1

Which of the following is the worst-case time complexity of a binary search, expressed in Big O notation?

- $\mathcal{O}(n \log n)$
- $\mathcal{O}(\log n)$
- $\mathcal{O}(n^2)$
- $\mathcal{O}(n)$

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Merge sort: time complexity

Challenge 1

Which of the following is the worst-case time complexity of a merge sort, expressed in Big O notation?

- $\mathcal{O}(\log n)$
- $\mathcal{O}(n)$
- $\mathcal{O}(n^2)$
- $\mathcal{O}(n \log n)$

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Big O: rank complexities

Challenge 2

Put the following Big O expressions into the order of increasing complexity.

Available items

- $O(n!)$
- $O(1)$
- $O(\log n)$
- $O(n \log n)$
- $O(2^n)$
- $O(n^2)$
- $O(n)$

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Intractable problem 1

Practice 2

What is an intractable problem?

- A problem for which no algorithm can be designed to produce a correct solution for all possible inputs in finite time.
- A problem that can be solved efficiently, typically in polynomial time less.
- A problem that cannot be solved efficiently, typically requiring exponential or factorial time.
- The undecidable problem of determining whether a given program will eventually halt or run forever on a given input.

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Raspberry Pi Foundation

Big O: determine expression 2

Challenge 2

The number of operations (steps) of an algorithm is given below, where n is the size of the input:

$5n^2 + \log_2 n + 2^n + 18$

How would the time complexity of this algorithm be expressed in **Big O notation**?

Type your answer in the format $\mathcal{O}(n)$, replacing n with the relevant term. If you want to use an exponent, use the ^ symbol, for example, **O(n^2)**.

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Binary search: trace

Challenge 2

The following list of items is stored in an array:

items							
index	0	1	2	3	4	5	6
value	2	4	5	8	12	15	18

The following algorithm has been coded and will be used to search for the number 12 as highlighted in the array.

Pseudocode

```
FUNCTION binary_search(items, search_item)
    found = False
    found_index = -1
    first = 0
    last = LEN(items) - 1

    WHILE first <= last AND found == False
        midpoint = (first + last) DIV 2
        IF items[midpoint] == search_item THEN
            found_index = midpoint
            found = True
        ELSEIF items[midpoint] < search_item THEN
            first = midpoint + 1
        ELSE
            last = midpoint - 1
        ENDIF
    ENDWHILE

    RETURN found_index
ENDFUNCTION
```

Complete the trace table for the algorithm. The first pass has already been filled for you.

first	last	midpoint	items [midpoint]	found_ index	found
0	6	3	8	−1	False
					False
				4	True

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Binary search: Trace (recursive)

Challenge 2

The following list of items is stored in an array:

items								
index	0	1	2	3	4	5	6	7
value	1	4	8	9	10	12	14	15

The following algorithm has been coded and will be used to search for the number 8 as highlighted in the array.

Pseudocode

```
FUNCTION binary_search(items, search_item, first, last)
    IF first > last THEN
        RETURN -1
    ELSE
        midpoint = (first + last) DIV 2
        IF items[midpoint] == search_item THEN
            RETURN midpoint
        ELSEIF search_item > items[midpoint] THEN
            first = midpoint + 1
            RETURN binary_search(items, search_item, first, last)
        ELSE
            last = midpoint - 1
            RETURN binary_search(items, search_item, first, last)
        ENDIF
    ENDIF
ENDFUNCTION
```

Part A Missing arguments

Complete the missing arguments of the initial function call when searching for the value 8.

Enter only the numerical (integer) value for each missing argument.

```
test_items = [1, 4, 8, 9, 10, 12, 14, 15]

index = binary_search(test_items, ____, ____, ____)
```

Part B Complete trace table

Complete the trace table for the algorithm. The first pass has already been filled in for you.

Call	`first`	`last`	`midpoint`	`items [midpoint]`	Line where return statement is executed
1	0	7	3	9	13
2					
3					

The **Call** column keeps track of how many calls the recursive algorithm has made to the `binary_search` function.

The **final column** is used to enter the line number from the algorithm where the next return statement is run e.g. in Call 1, neither the condition `items[midpoint] == search_item` nor `search_item > items[midpoint]` are true since 8 is less than 9, so the return statement on line 13 is executed next.

Part C Determine the outcome

What will happen if the function is called and the argument for `search_item` is the value 11?

- The program will cause a runtime error
- The program will output a message that the value hasn't been found
- The program will return the value -1
- The program will cause a stack overflow and crash

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Bubble sort: fix

Challenge 1

Kofi has written a program to sort an array of data into **descending** order. The algorithm is a version of bubble sort and is shown in pseudocode below.

Pseudocode

```
// Initialise variables
items = [28, 40, 21, 25, 30, 27, 25]
num_items = LEN(items)
temp = 0
pass_number = 1
swapped = False

// Continue while swaps have been made and there are more passes to evaluate
WHILE swapped == True AND pass_number <= num_items - 1
   swapped = False
   FOR index = 0 TO num_items - 2
       // Check if items are out of order
       IF items[index] < items[index + 1]
           // Swap items
           temp = items[index]
           items[index] = items[index + 1]
           items[index + 1] = temp
           swapped = True
        ENDIF
    NEXT index
    pass_number = pass_number + 1
ENDWHILE
```

Part A

There is a problem with the algorithm and it will not sort the data correctly. What is the problem?

- The `items` are being compared in the wrong order.
- The outer loop is running insufficient times.
- The inner loop is running too many times.
- The variable `swapped` has been initialised incorrectly.

Part B

Kofi has corrected the error in the pseudocode.

What is the **total number of swaps** required to sort the array `items` into descending order?

Enter your answer as a **number**.

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Recursion: purpose of subroutine 2

Challenge 2

A recursive subroutine has been written in high-level pseudocode, as follows:

Pseudocode

```
PROCEDURE traverse(tree)
    IF tree != Null THEN
        traverse(tree.RightChild())
        PRINT(tree.node.get_value())
        traverse(tree.LeftChild())
    ENDIF
ENDPROCEDURE
```

What **variant** of a standard algorithm does it implement?

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

Raspberry Pi Foundation

Features of recursion

Challenge 2

LLM marked question

Recursion is a technique often used in programming to solve complex problems by breaking them down into smaller, similar subproblems.

30 attempts remaining today

State two features of recursion in programming.

[2 marks]

Quiz:
STEM SMART Computer Science Week 46

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.